Automation Practice Blogspot Report:-

1.Introduction:-

This project is a web automation testing framework developed using **Selenium WebDriver**, **Java**, **TestNG**, and **Maven** following the **Page Object Model (POM)** design pattern. The framework automates multiple functionalities of the BlogSpot test application, including form handling, date pickers, file uploads, web tables, alerts, windows/tabs, Shadow DOM, drag-and-drop, sliders, and broken links. It also implements **data-driven testing** using Excel files for dynamic test input management. To improve reliability, **assertions** are used for validation and **TestNG listeners** are integrated for execution tracking and reporting. The framework captures **screenshots automatically on test failure** to support debugging and analysis. Additionally, **TestNG groups** are used to organize smoke and regression test execution efficiently. Overall, this project demonstrates a structured and maintainable automation framework suitable for real-world testing scenarios.

2. Technologies Used:-

- Java
- Selenium WebDriver
- TestNG
- Maven
- Apache POI (Data-driven from excel file)
- Page Object Model (POM)
- Git & GitHub
- Jenkins (CI/CD Pipeline)

3. Framework Structure:-

- Base Package
- Pages Package
- Utilities Package
- Testcases Package
- Listeners Package
- Reports Package (for extent report generation)

- Screenshots Folder (takes Screenshot on failures)
- Data-driven using excel file (ExcelUtils.java)
- pom.xml
- testng.xml
- Jenkinsfile

4. Automated Test Flow:-

I. Launch https://testautomationpractice.blogspot.com on Chrome Browser

II. Automated text fields(read data from excel file), radio button, check box

III. Automated different formats dropdowns

IV. Automated different formats of date pickers

V. Automated single file upload, multiple files upload, pagination web table

VI. Automated form sections, Wikipedia search field, dynamic button

VII. Automated different types of alerts, new tab, popup windows

VIII. Automated different types of actions, scrolling dropdown

IX. Automated laptop link, broken link and ShadowDOM

X. Listens to test events like TestStart, TestSuccess, TestFailure, TestSkipped using TestListener class

XI. Validate every test case from test class

XII. Capture Screenshot on failures

5. Test Execution Summary:-

Test Data Used

Test Scenario	Test Data
Form Validation	Name: MD ASRAFUL, Email: example@gmail.com, Phone: 2546987513, Address: XYZ Road, ABC
Gender Selection	Male

Test Scenario	Test Data
Day Selection	Saturday, Sunday
Country Dropdown	India
Colors Dropdown	Green
Animals Dropdown	Lion
Date Picker 1	19/07/2002
Date Picker 2	11 March 2016
Date Range	Start: 03/01/2003, End: 05/06/2026
Single File Upload	ABC.txt
Multiple File Upload	f.txt, file2.txt, t.txt
Dynamic Input Section	Hello, Hii, Bye
Wikipedia Search	Java
Prompt Alert Input	Harry Potter
Shadow DOM Input	Hii
Shadow DOM File Upload	ABC.txt

Total Tests Executed:- 10

Passed:- 10

Failed:- 0

Skipped:- 0

Pass Percentage:- 100%

6. Execution Log Highlights:-

→ Test suite execution started using TestNG through testng.xml

→ Browser launched successfully in Google Chrome using WebDriverManager

→ Test data fetched dynamically from Excel file (testdata.xlsx) for data-driven testing

→ Form details entered and validated successfully using assertions

→ Dropdown values (country, colors, animals) selected and verified

→ Date pickers and date range fields populated and validated

→ Single and multiple file upload operations executed successfully

→ Pagination web table rows selected and validated

→ Dynamic input form sections filled and verified

→ Wikipedia search functionality executed and results displayed successfully

→ Simple alert, confirm alert, and prompt alert handled successfully

→ Popup window and new tab opened, validated, and closed properly

→ Mouse hover, double click, drag-and-drop, slider, and scrollable dropdown actions performed successfully

→ External link navigation validated on Apple Inc. website and returned to home page

→ Broken link navigation validated by confirming 404 error page and returning back

→ Shadow DOM elements accessed successfully, including text input, checkbox selection, and file upload

→ Assertions executed across modules for validation of expected results

→ Custom TestNG listener captured execution status (Start, Pass, Fail, Skip)

→ Screenshots automatically captured on test failures for debugging

→ Reports generated successfully:
• TestNG Report (index.html)
• Emailable Report (emailable-report.html)
• Extent Report (ExtentReport.html)

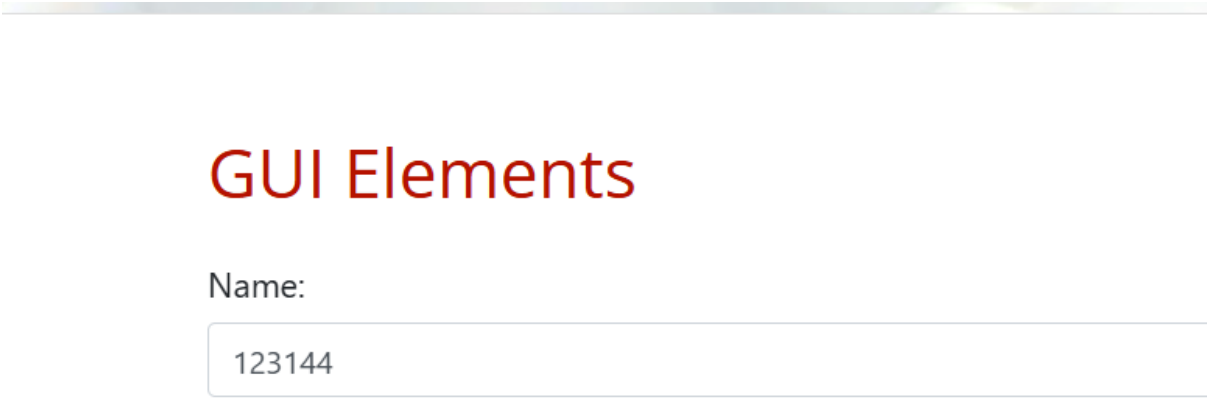
→ Smoke and Regression test groups configured for selective execution

→ Final Execution Summary:
Passed = 10
Failed = 0
Skipped = 0

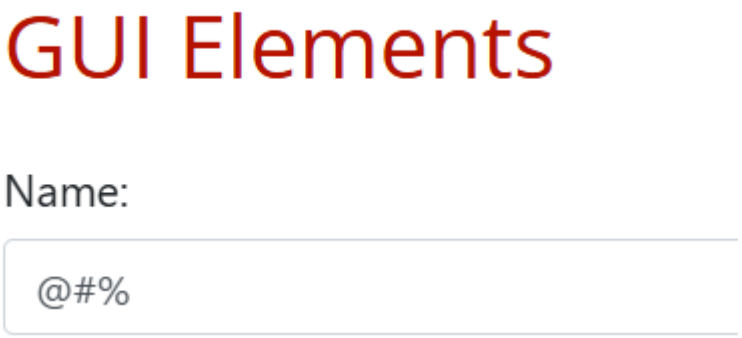
→ Test suite execution completed successfully

7. Defects Identified During Testing:-

a. Name Field Accepts Numeric Values Without Validation

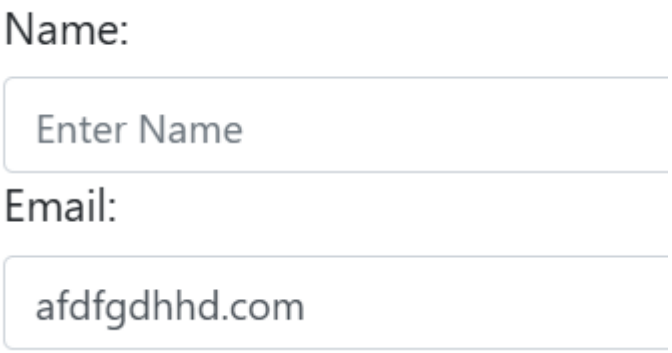


b. Name Field Accepts Special Characters Without Validation

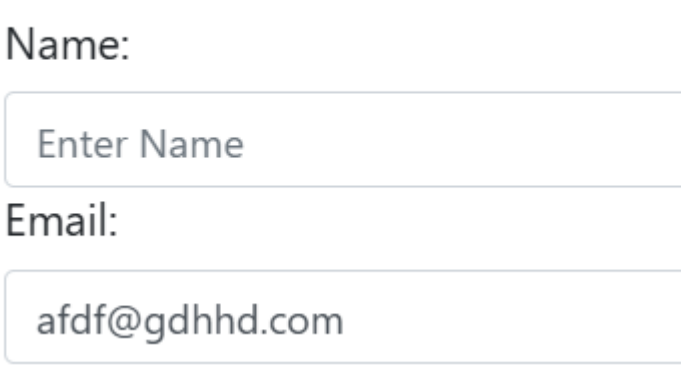


c. Name Field Accepts Alphanumeric Characters Without Validation

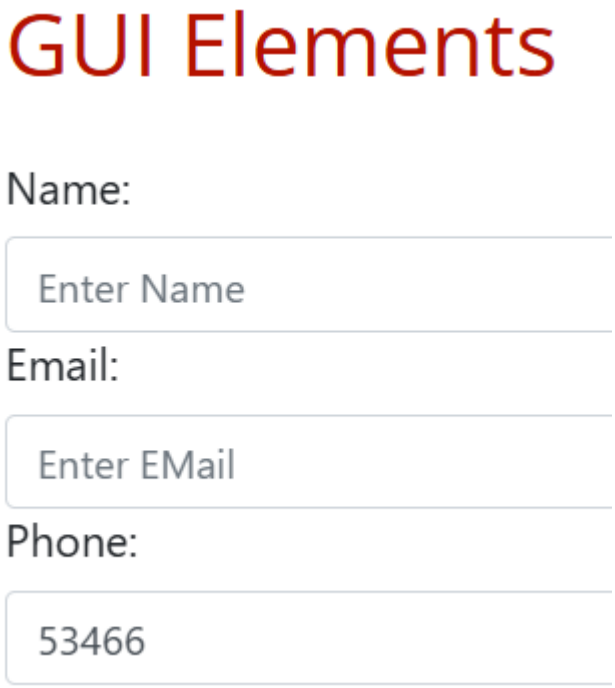
d. Email Field Accepts Invalid Email Format Without Validation



e. Email Field Accepts Invalid Domain Format Without Validation

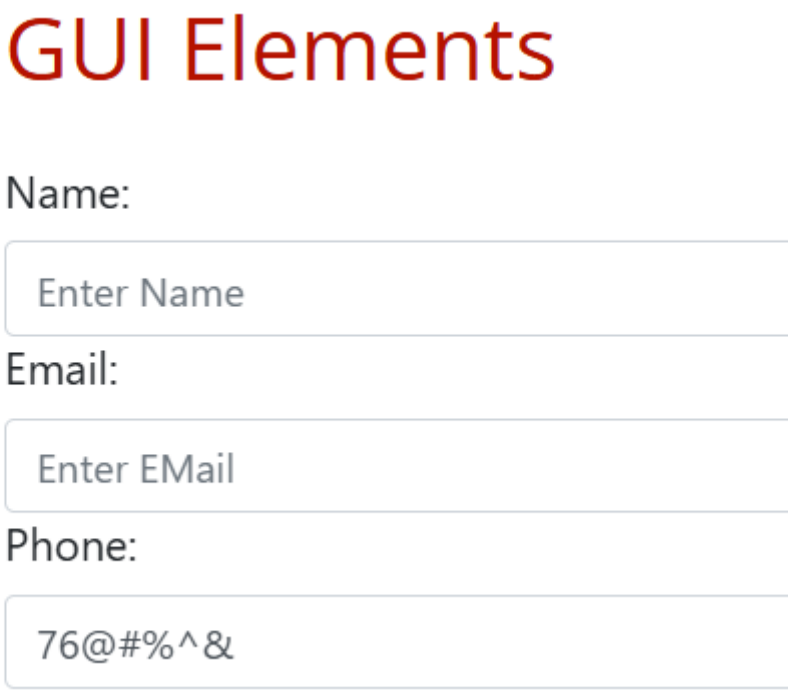


f. Mobile Number Field Accepts Less Than 10 Digits Without Validation

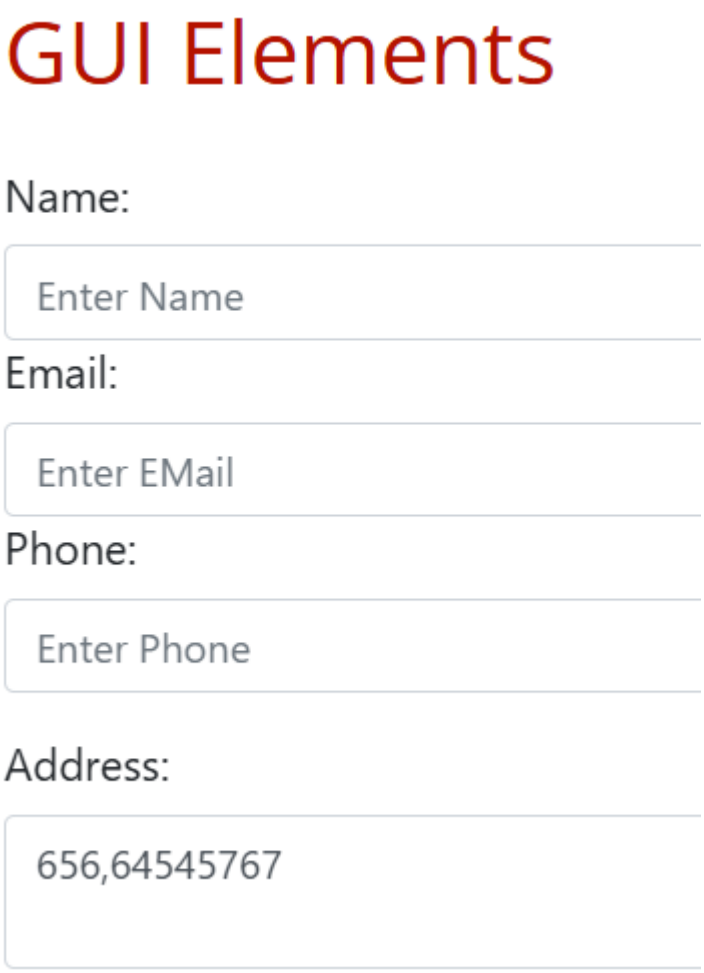


g. Mobile Number Field Accepts Alphanumeric Characters Without Validation

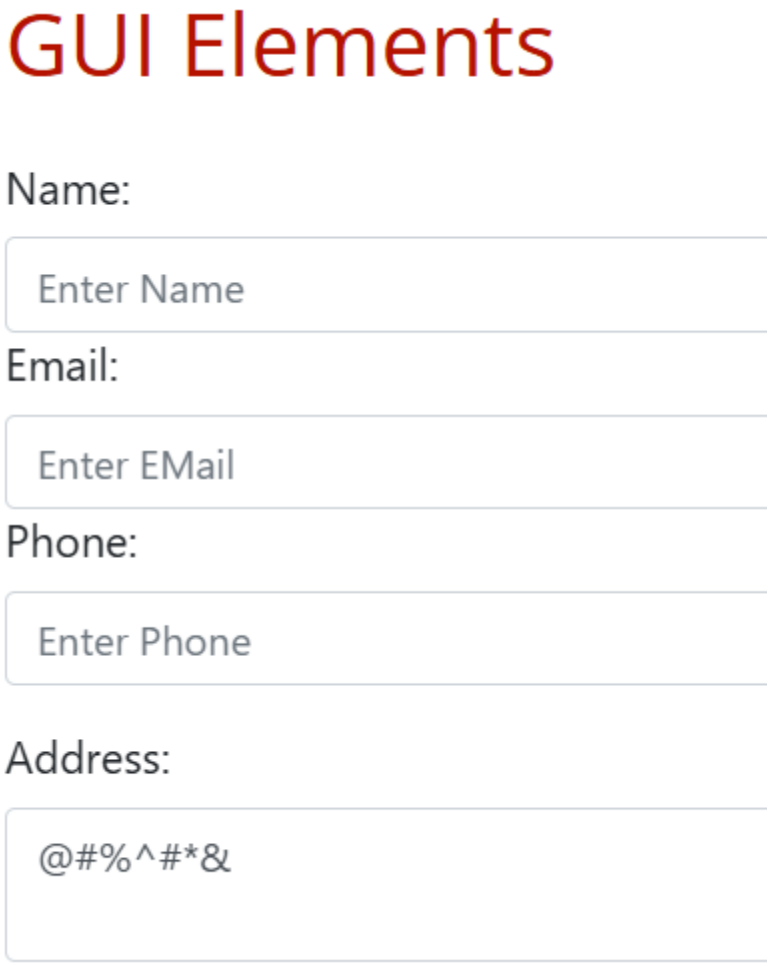
h. Mobile Number Field Accepts Special Characters Without Validation



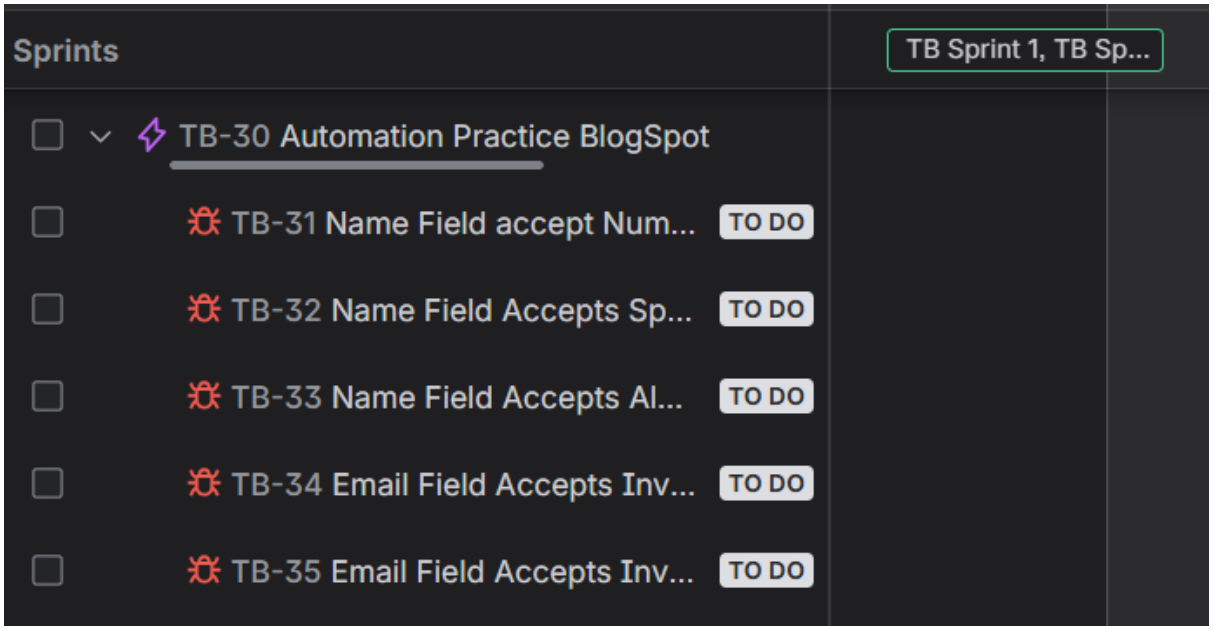
i.Address Field Accepts Numeric-Only Input Without Validation

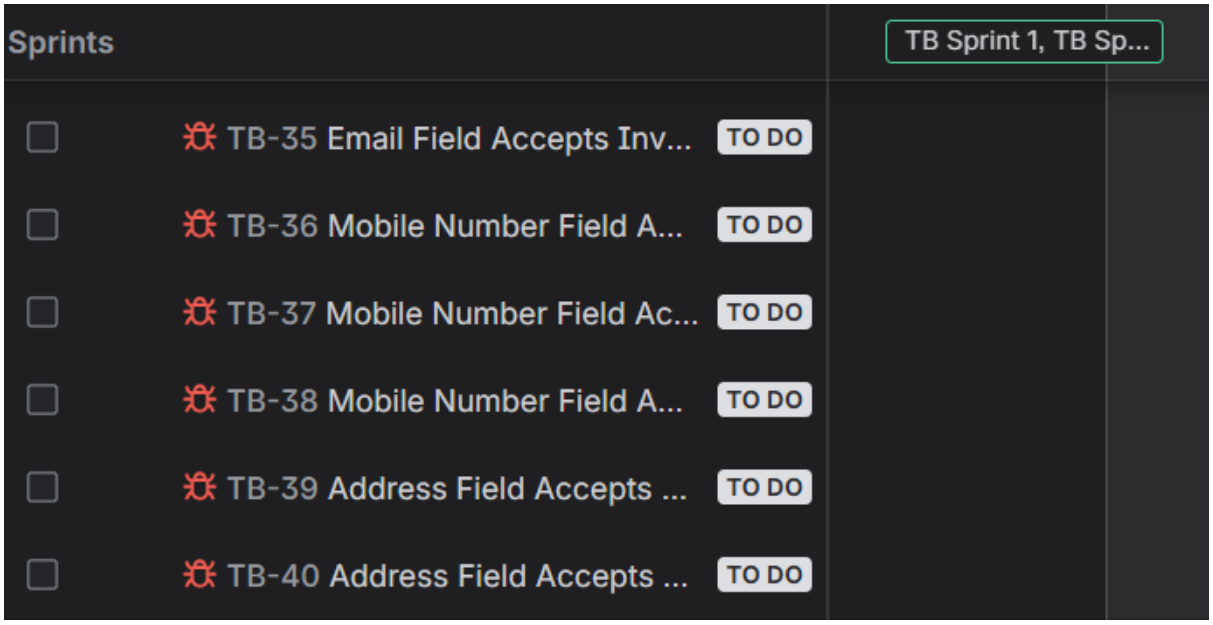


j. Address Field Accepts Special Characters Only Without Validation



8. Jira Defect Screenshots:-





Name Field accept Numeric Values without Validation.

To Do

Improve Bug

Description

The Name field allows users to enter numeric values (e.g., `12345`) and successfully submit the form without displaying any validation error. The field should accept only alphabetic characters and valid name formats.

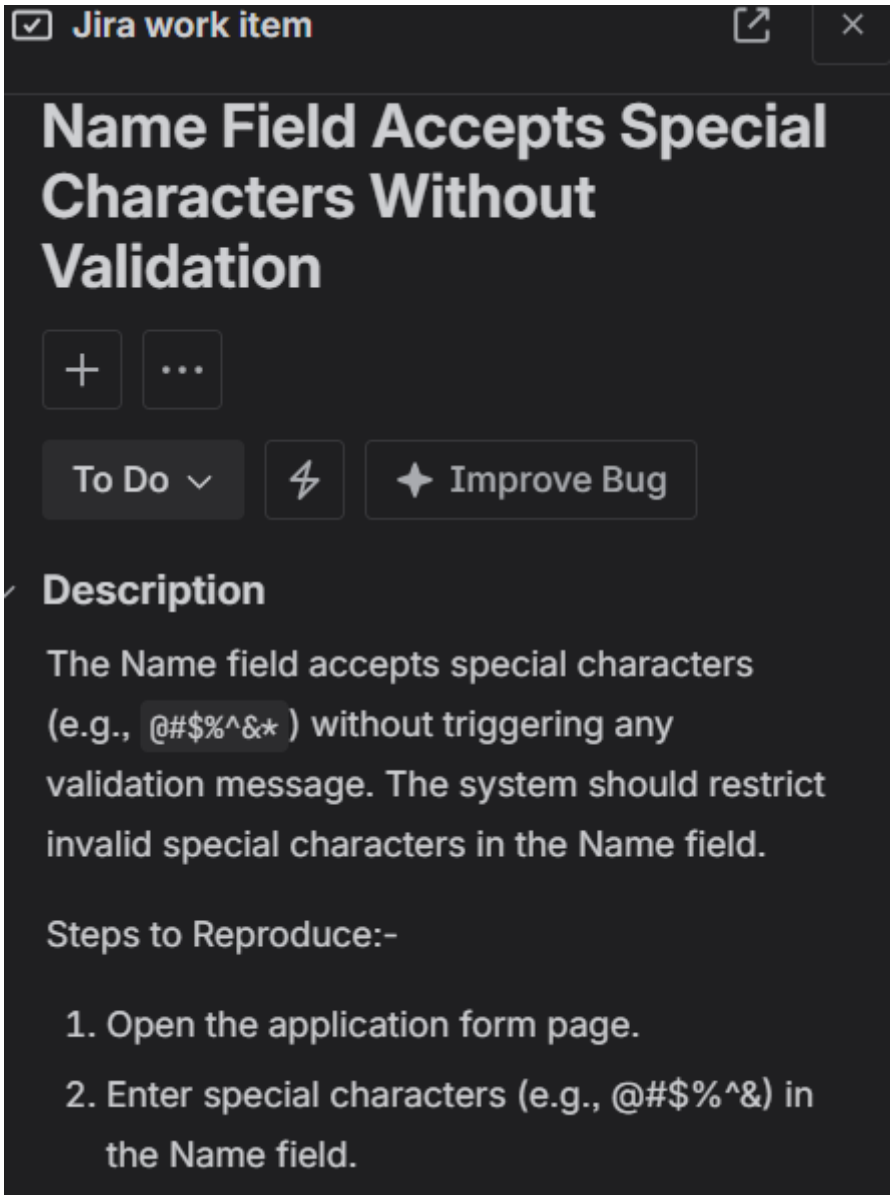
Steps to Reproduce:-

1. Open the application form page.
2. Enter numeric values (e.g., 12345) in the Name field.
3. Fill all other mandatory fields with valid

3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when numeric values are entered in the Name field.



2. Enter special characters (e.g., @#$%^&) in the Name field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when special characters are entered in the Name field.

Name Field Accepts Alphanumeric Characters Without Validation

To Do | Improve Bug

Description

The Name field accepts a combination of letters and numbers (e.g., `John123`) without validation. The field should allow only valid name characters as per business requirements.

Steps to Reproduce:-

1. Open the application form page.
2. Enter alphanumeric values (e.g., John123) in the Name field.

2. Enter alphanumeric values (e.g., John123) in the Name field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when alphanumeric values are entered in the Name field.

Email Field Accepts Invalid Email Format Without Validation

To Do | Improve Bug

Description

The Email field allows invalid email formats (e.g., `useremail.com`, `user@`) to be submitted without displaying an error message. Proper email format validation is missing.

Steps to Reproduce:-

1. Open the application form page.
2. Enter an invalid email address (e.g., testemail.com) in the Email field.
2. Enter an invalid email address (e.g., testemail.com) in the Email field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when an invalid email format is entered.

Email Field Accepts Invalid Domain Format Without Validation

To Do | Improve Bug

Description

The Email field accepts invalid domain formats (e.g., `user@domain`, `user@.com`) without validation. The system should validate the domain structure before accepting the email address.

Steps to Reproduce:-

1. Open the application form page.
2. Enter an email address with an invalid domain (e.g., test@domain) in the Email

2. Enter an email address with an invalid domain (e.g., test@domain) in the Email field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when an invalid domain format is entered.

Mobile Number Field Accepts Less Than 10 Digits Without Validation

To Do | Improve Bug

Description

The Mobile Number field allows numbers containing fewer than 10 digits (e.g., `123456789`) to be submitted without displaying a validation error. The field should enforce the required digit length.

Steps to Reproduce:-

1. Open the application form page.
2. Enter a mobile number with less than 10 digits (e.g., 123456789).
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when a mobile number contains fewer than 10 digits.

Mobile Number Field Accepts Alphanumeric Characters Without Validation

To Do | Improve Bug

Description

The Mobile Number field accepts alphanumeric input (e.g., `98765ABC12`) without validation. The field should only allow numeric values.

Steps to Reproduce:-

1. Open the application form page.
2. Enter alphanumeric values (e.g., 98765ABC12) in the Mobile Number field.
2. Enter alphanumeric values (e.g., 98765ABC12) in the Mobile Number field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when alphanumeric characters are entered in the Mobile Number field.

Mobile Number Field Accepts Special Characters Without Validation

To Do | Improve Bug

Description

The Mobile Number field accepts special characters (e.g., `98765@#$%1`) without displaying any validation error. The field should restrict special character input.

Steps to Reproduce:-

1. Open the application form page.
2. Enter special characters (e.g., @#$%^&) in the Mobile Number field.

2. Enter special characters (e.g., @#$%^&) in the Mobile Number field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when special characters are entered in the Mobile Number field.

Address Field Accepts Numeric-Only Input Without Validation

To Do

Improve Bug

Description

The Address field allows numeric-only values (e.g., `123456`) to be submitted without validation. The field should require meaningful address information containing valid address components.

Steps to Reproduce:-

1. Open the application form page.
2. Enter numeric values only (e.g., 123456) in the Address field.

2. Enter numeric values only (e.g., 123456) in the Address field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

Expected Result:-

The system should display a validation message and prevent submission when numeric-only values are entered in the Address field.

Address Field Accepts Special Characters Only Without Validation

To Do | Improve Bug

Description

The Address field accepts input consisting solely of special characters (e.g., `@#$%^&*`) without displaying a validation error. The system should prevent invalid address entries and enforce meaningful address content.

Steps to Reproduce:-

1. Open the application form page.
2. Enter special characters only (e.g., @#$%^&) in the Address field.

2. Enter special characters only (e.g., @#$%^&) in the Address field.
3. Fill all other mandatory fields with valid data.
4. Click on Submit.

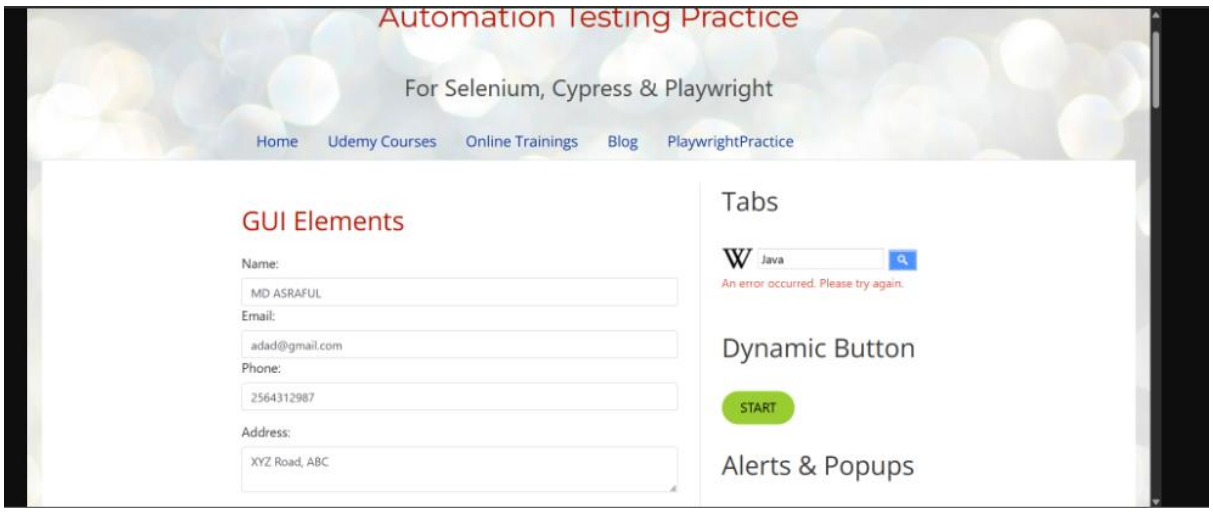
Expected Result:-

The system should display a validation message and prevent submission when special characters only are entered in the Address field.

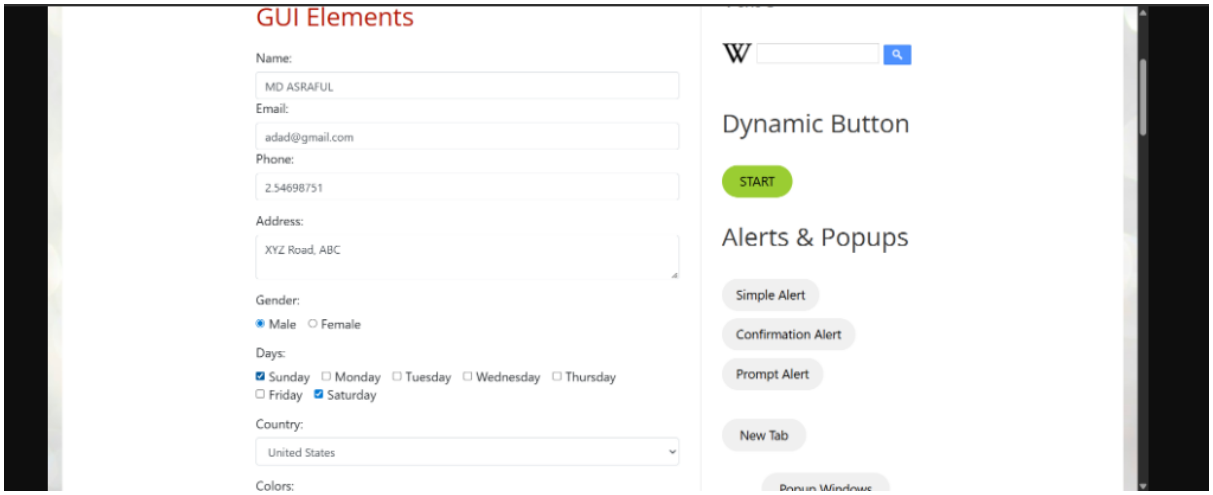
9. Screenshots & Evidence:-

Screenshots are captured automatically during execution and stored in the screenshots folder when test failure occurs.

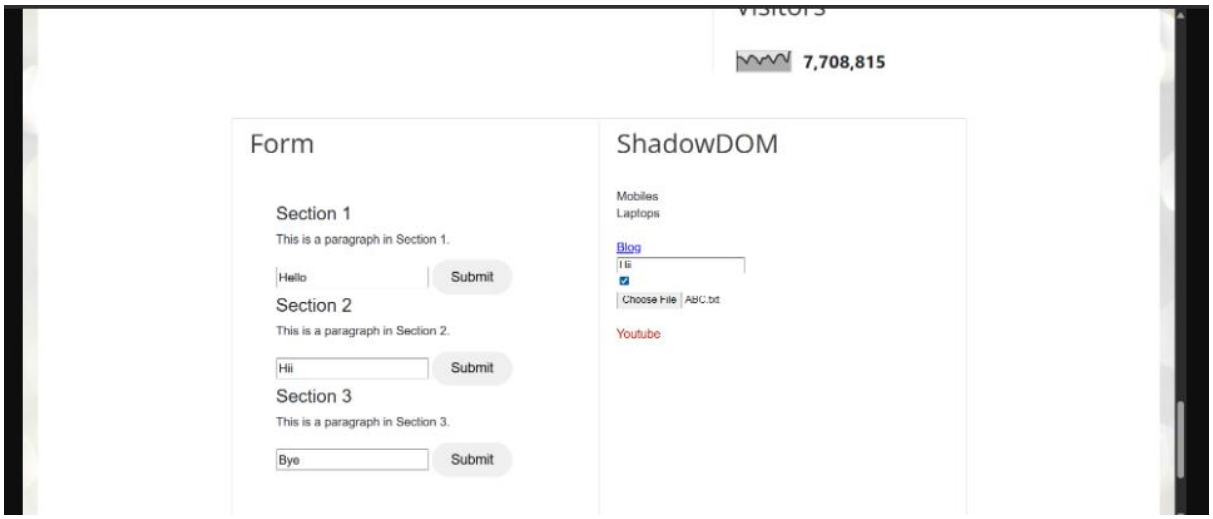
a.wikipediaTest.png:-



b.formDetailsTest.png:-



c.actionsLaptopBrokenLinksAndShadowDomTest.png:-

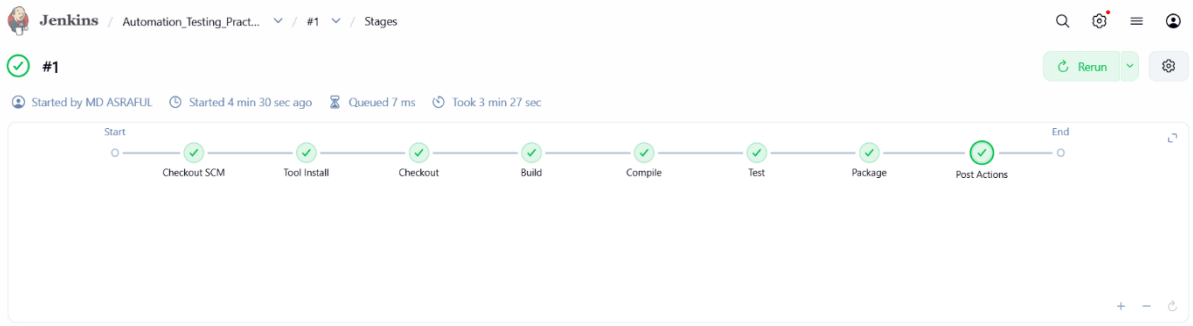


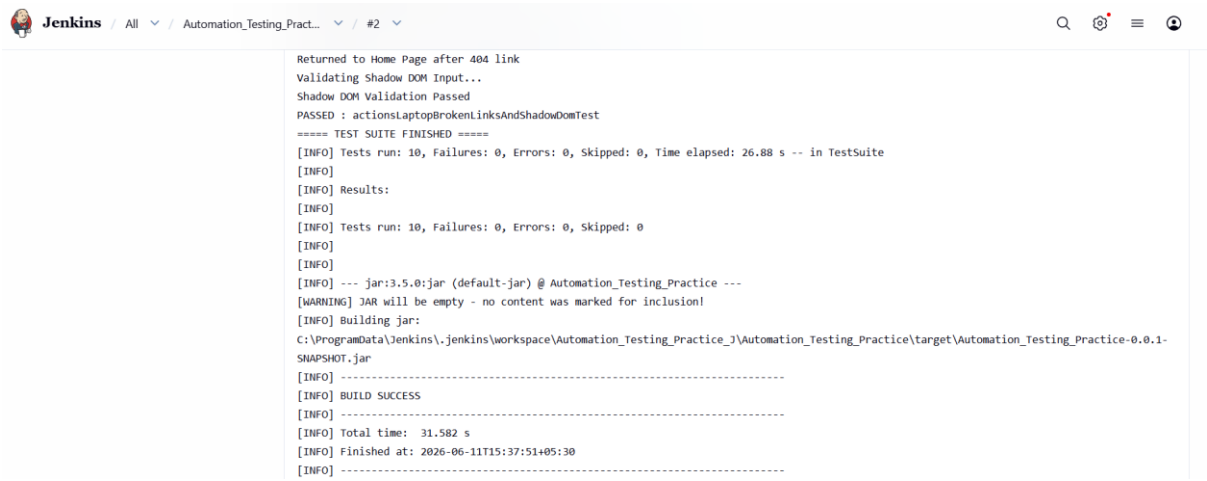
Command – mvn clean test:-

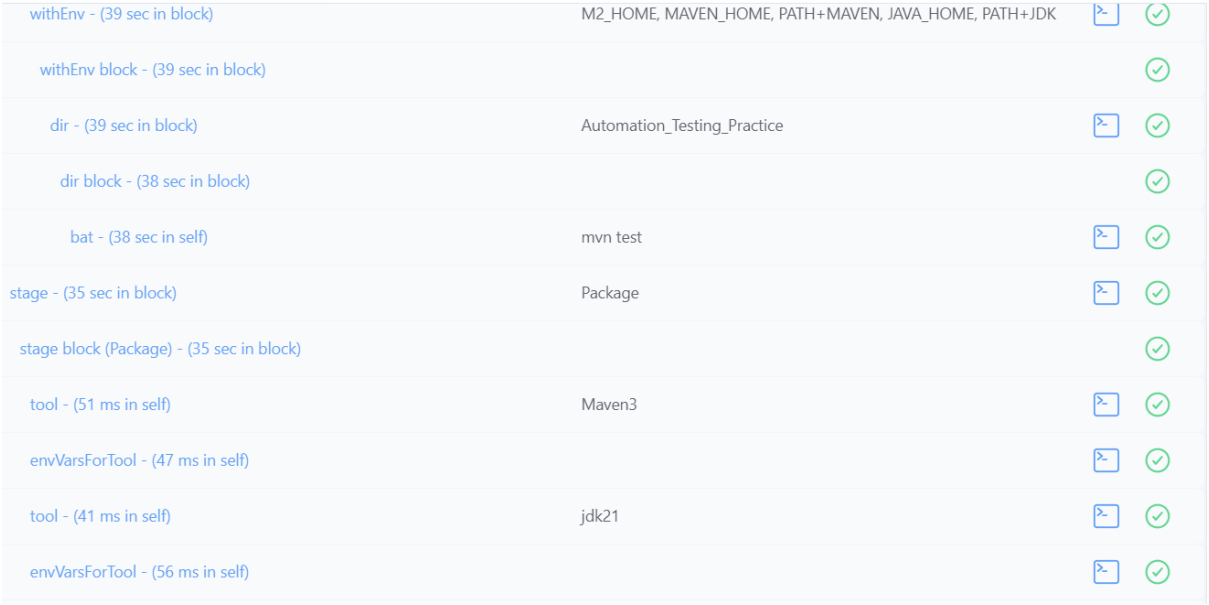
```
===== TEST SUITE FINISHED =====
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 21.17 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  26.142 s
[INFO] Finished at: 2026-06-11T13:01:38+05:30
[INFO] ------------------------------------------------------------------------
```

10. CI/CD Pipeline using Jenkins:-

The Project is prepared for CI/CD Pipeline through Jenkins. Maven commands can be executed through Jenkins jobs.

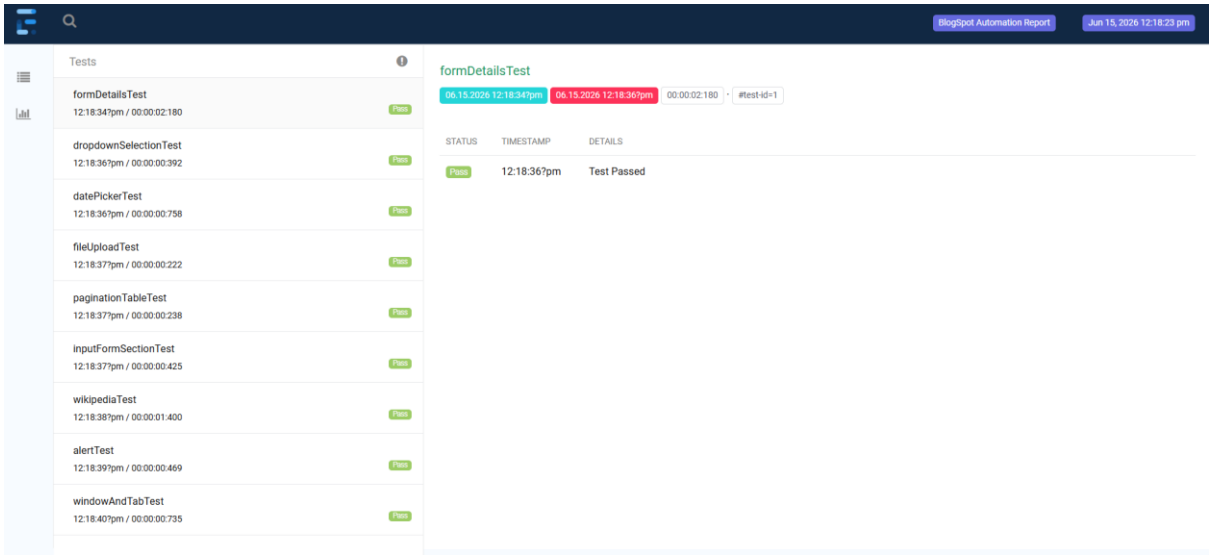


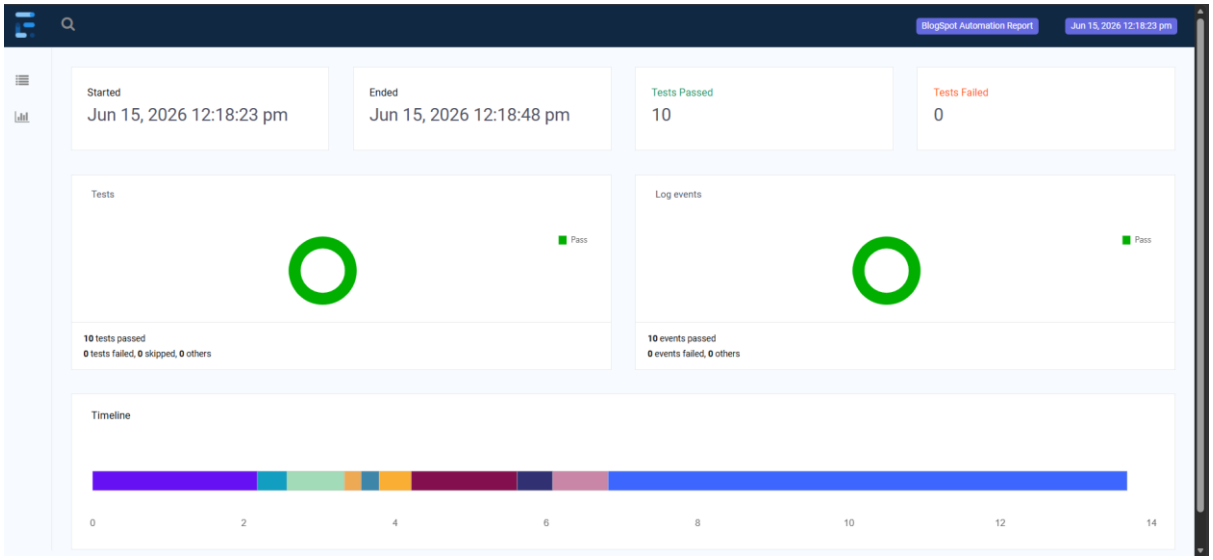




11.Reports:-

a.ExtentReport.html:-





b.emailable-report.html:-

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	Included Groups	Excluded Groups
Suite							
BlogSpotTests	10	0	0	0	25,067		

Class	Method	Start	Time (ms)
Suite			
BlogSpotTests — passed			
tests.BlogSpotTest	actionsLaptopBrokenLinksAndShadowDomTest	1781506121030	6874
	alertTest	1781506119814	468
	datePickerTest	1781506116728	768
	dropdownSelectionTest	1781506116334	394
	fileUploadTest	1781506117502	224
	formDetailsTest	1781506114126	2183
	inputFormSectionTest	1781506117969	429
	paginationTableTest	1781506117726	238
	wikipediaTest	1781506118405	1402
	windowAndTabTest	1781506120291	734

BlogSpotTests

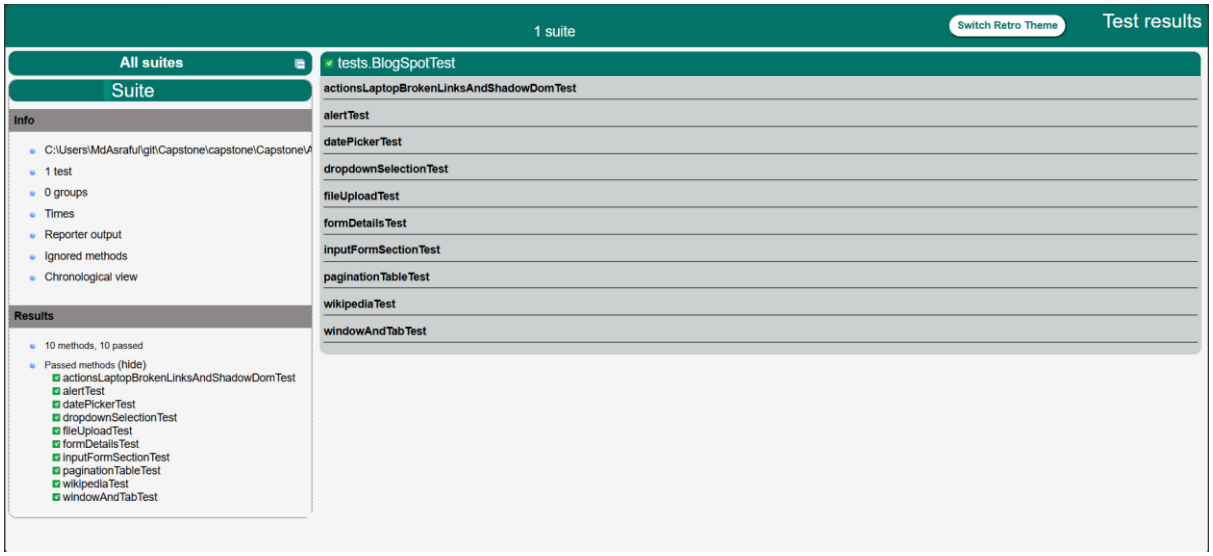
tests.BlogSpotTest#actionsLaptopBrokenLinksAndShadowDomTest

back to summary

tests.BlogSpotTest#alertTest

back to summary

c.index.html:-



12. Conclusion:-

This automation testing project successfully demonstrates the implementation of a robust and scalable web automation framework using Selenium WebDriver, Java, and TestNG. By applying the Page Object Model (POM) design pattern, the framework achieved better code

reusability, maintainability, and readability. Multiple real-world functionalities such as forms, alerts, file uploads, web tables, Shadow DOM, windows/tabs, and advanced user actions were automated and validated effectively. The integration of assertions, listeners, screenshot capture on failure, test grouping, and Extent Reports enhanced the reliability and debugging capability of the framework. Data-driven testing using Excel further improved test flexibility and reduced hardcoded inputs. Overall, this project strengthened practical knowledge of automation testing and reflects an industry-level approach to building a maintainable testing framework.

Done By – MD ASRAFUL (SDET B-3),

Guided By – Vaishali Ma’am (Trainer)